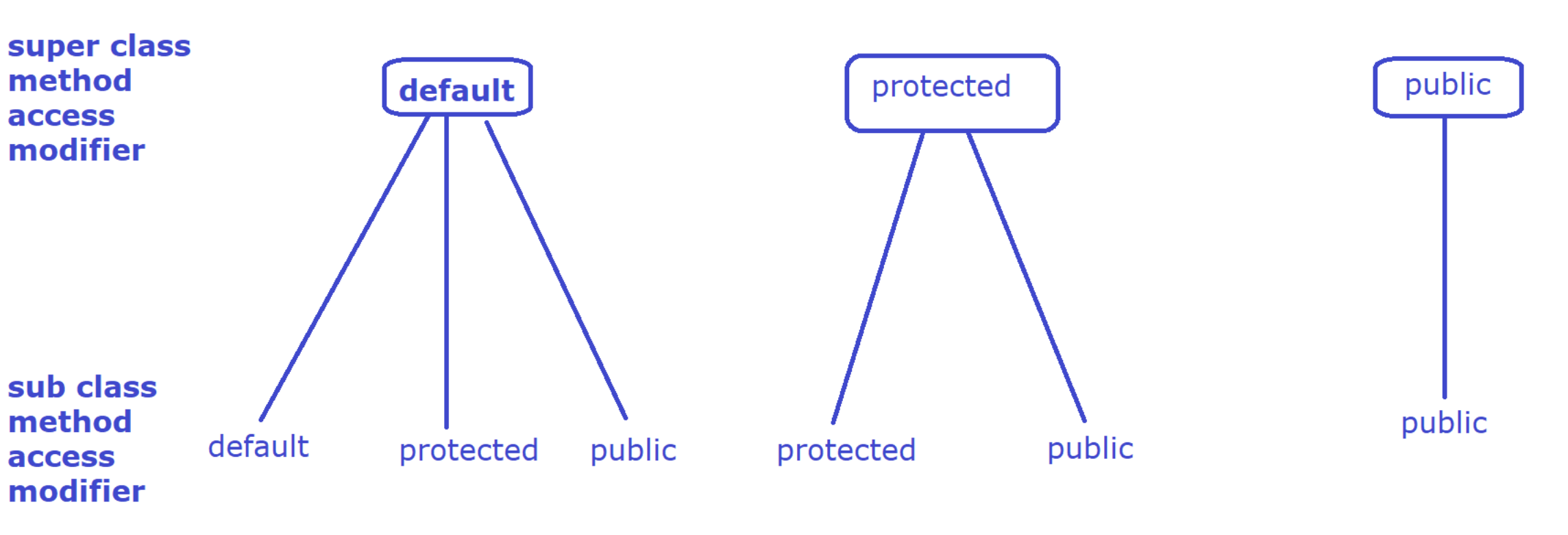
Role of access modifier while overriding a method :

In terms of accessibility, public is greater than protected, protected is greater than default (public > protected > default)
[default < protected < public]

While overriding a method, we can't reduce the visibility of the overridden method otherwise JVM cannot access the method for execution that is the reason if we try to reduce the visibility then we will get compilation error.
[We can increase the visibility]

Note :- private method is not availble (visible) in sub class so it is not the part of method overriding.




```
//Program

class A
{
    public void show()
    {
    }
}
class B extends A
{
    @Override
    protected void show()  //error [reducing the visibility]
    {
    }
}
public class Visiblity
{
    public static void main(String[] args)
    {
        A a1 = new B();
        a1.show();
    }
}
```

IQ:

```
package com.ravi.method_overriding;

class Person
{
    protected int age = 34;

    public int getAge()
    {
        return this.age;
    }

    public void printAge()
    {
        IO.println(this.getAge());
    }
}

class Student extends Person
{
   protected int age = 18;

   @Override
   public int getAge()
   {
       return this.age;
   }
}

public class IQ
{
    public static void main(String[] args)
    {
       Person p = new Student();
       p.printAge();   //18
    }

}
```

In the above program, Output is : 18
What if, we want to get the output 34, What changes are required ?

```
package com.ravi.method_overriding;

class Person
{
    protected int age = 34;

    public int getAge()
    {
        return this.age;
    }

    public void printAge()
    {
        IO.println(this.getAge());
    }
}

class Student extends Person
{
   protected int age = 18;

   @Override
   public int getAge()
   {
       return this.age;
   }

   @Override
   public void printAge()
    {
        IO.println(super.getAge());
    }
}

public class IQ
{
    public static void main(String[] args)
    {
       Person p = new Student();
       p.printAge();
    }

}
```

Method overloading is possible in super and sub class as shown in the program below

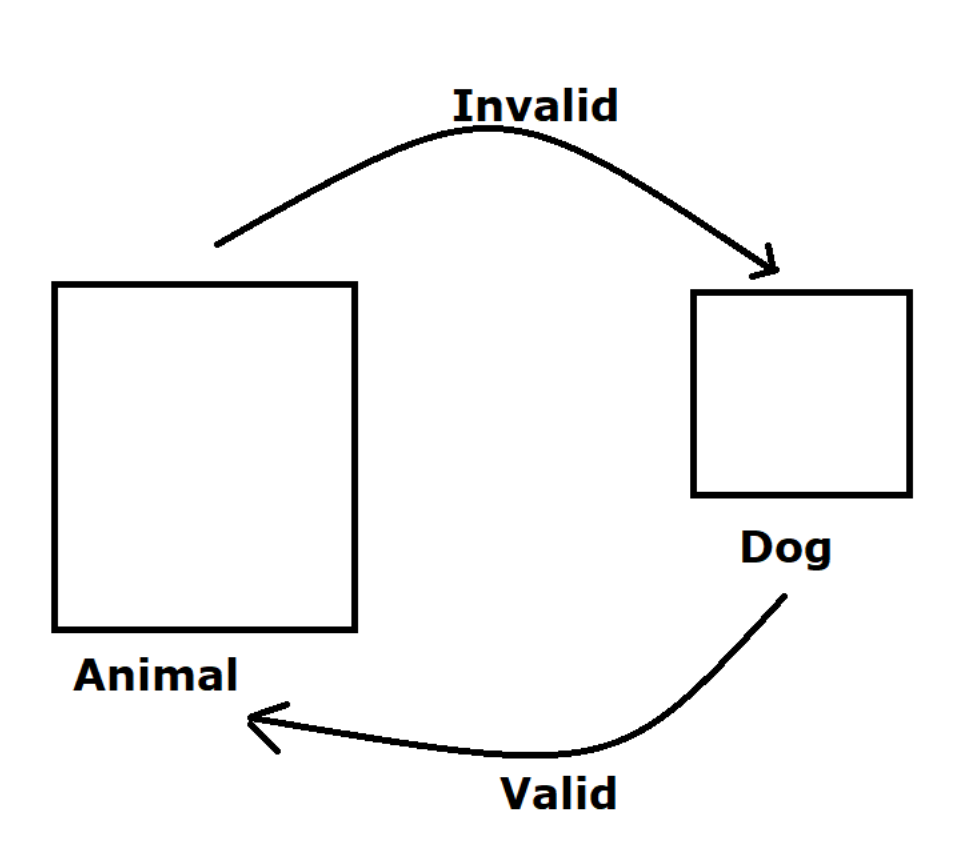
```
package com.ravi.method_overriding;

class Bird
{
    public void roam()
    {
        IO.println("Generic Bird is roaming");
    }

    public void fly()
    {
        IO.println("Generic Bird is flying");
    }
}
class Parrot extends Bird
{
    //Overriding
    public void roam()
    {
        IO.println("Parrot Bird is roaming");
    }

    //Overloading
    public int fly(double height)
    {
        IO.println("Parrot Bird is flying with "+height+" feet height");
        return 0;
    }
}
public class IQ
{
    public static void main(String[] args)
    {
      Parrot p = new Parrot();
      p.roam();
      p.fly(12.90);
    }

}
```

```
package com.ravi.method_overriding;

class Animal
{
}
class Horse extends Animal
{
}

class Overloading
{
    public void accept(Animal a)
    {
        IO.println("Animal version");
    }

    public void accept(Horse a)
    {
        IO.println("Horse version");
    }
}

public class IQ
{
    public static void main(String[] args)
    {
        Overloading overload = new Overloading();
        Animal a1 = new Animal();
        overload.accept(a1); //Animal Version

        Animal a2 = new Horse();
        overload.accept(a2);  //Animal Version


    }

}
```

Polymorphic behavior of Method while using Method Overriding :

```
package com.ravi.method_overriding;

class Vehicle
{
    public void run()
    {
        IO.println("Generic Vehicle is running");
    }
}
class Car extends Vehicle
{
    public void run()
    {
        IO.println("Car Vehicle is running");
    }
}

class Bike extends Vehicle
{
    public void run()
    {
        IO.println("Bike Vehicle is running");
    }
}


public class PolymorphicBehaviour
{
    public static void main(String[] args)
    {
        Vehicle v = new Car();
        vehicleCamp(v);

        v = new Bike();
        vehicleCamp(v);
    }

    public static void vehicleCamp(Vehicle vehicle)  //Loose coupling
    {
        vehicle.run();
    }

}
```

Downcasting :



Dog d = (Dog) new Animal();

* Here code will compile but at runtime JVM will generate a rutime execption i.e java.lang.ClassCastException

* Downcasting is used to call the specific method of a prticular class. [bark() method of Dog class]

* Downcasting is not possible without up-casting

* Downcasting is a technique to assign **sub class object to sub class reference variable using super class reference variable.**

Example :

```
Animal a = new Dog(); //Upcasting
        Dog d = (Dog) a;  //Downcasting
        d.sleep();
        d.bark();
```

```
//Program :
-------------
 class Animal
 {
     public void sleep()
     {
         System.out.println("Generaic Aninmal is sleeping here");
     }
 }
 class Dog extends Animal
 {
     public void sleep()
     {
         System.out.println("Dog Aninmal is sleeping here");
     }

     public void bark()
     {
         System.out.println("Dog is barking");
     }
 }


 public class MethodOverriding
 {
     public static void main(String[] args)
     {
         Animal a = new Dog(); //Upcasting
         Dog d = (Dog) a;  //Downcasting
         d.sleep();
         d.bark();
     }
 }

 /*
   bark();  : Javac ->  Dog type of ref is reqd
             JVM   -> Dog type of object

 */
```